

1 Latent Representation Probing for Toxic Comment Detection

1.1 Problem Statement

1.1.1 Executive Summary

Online toxicity—including hate speech, severe insults, harassment, and explicit language—degrades user experience, discourages constructive dialogue, and creates significant moderation challenges at scale. Traditional keyword filters and standard fine-tuned classifiers can miss subtle semantic context, while deploying a large language model for every inference can be computationally expensive and introduce latency.

This project investigates accurate, lightweight toxicity detection through latent feature representations and probe classification. The approach extracts internal activation vectors from a base language model and trains a lightweight classifier to distinguish toxic from non-toxic comments.

1.1.2 Background and Problem Context

Scale of moderation. Digital platforms process very large volumes of user-generated content, making fully manual moderation infeasible.

Contextual nuance. Toxic statements may be implicit, veiled, sarcastic, or dependent on surrounding context. Simple rules and keyword lists can therefore miss harmful content or over-flag benign language.

Efficiency requirements. Large generative models can provide strong semantic understanding, but running full-model inference for every comment can be costly in terms of compute, memory, and response time.

1.1.3 Objective and Technical Scope

The objective is to build a binary toxicity classifier that separates toxic comments from non-toxic comments.

The proposed workflow extracts latent internal activation vectors—such as hidden-state representations—from selected transformer layers of a base language model, for example Gemma. A lightweight probe, such as logistic regression implemented with scikit-learn, is then trained on those representations. The base model can remain frozen while the probe learns a task-specific decision boundary.

The intended users include online platform moderators, trust-and-safety teams, and AI-safety researchers studying representation engineering for content moderation. Exact model size, layer selection, dataset split, and evaluation metrics should be populated from the final notebook outputs before formal benchmarking claims are made.

1.1.4 Success Criteria

A successful implementation should provide a reproducible extraction-and-probing pipeline, report standard classification metrics, characterize false positives and false negatives, and document operating thresholds suitable for human review or automated action.

1.1.5 Key Design Principle

The system should be treated as a decision-support component rather than an unquestionable authority. High-impact moderation decisions should incorporate calibrated thresholds, monitoring, and human oversight.

Prepared from the supplied project outline; unspecified experimental values remain intentionally open for notebook-specific completion.